Docker Cheatsheet

Images, containers, volumes and Compose. Every command here is from the hands-on tutorial.

CODE BASICS

Everyday loop

QUICK START

```
docker build -t api:v1 .
docker run -d -p 8000:8000 --name api api:v1
curl http://localhost:8000/health
docker logs -f api
docker stop api && docker rm api
```

Images

BUILD · SHIP

`docker build -t api:v1 .`	Build the image
`docker build -f Dockerfile.gpu -t api:gpu .`	Use another Dockerfile
`docker build --target builder .`	Stop at a stage
`docker images`	List images
`docker pull python:3.12-slim`	Download an image
`docker tag api:v1 me/api:v1`	Add a name
`docker push me/api:v1`	Upload to registry
`docker history api:v1`	Layers and sizes
`docker rmi api:v1`	Remove an image

Containers

RUN · DEBUG

`docker run -d -p 8000:8000 --name api api:v1`	Run in background
`docker ps / docker ps -a`	Running / all
`docker stop api / start api`	Stop / start
`docker restart api`	Stop, then start
`docker logs -f api`	Follow logs
`docker exec -it api sh`	Shell inside
`docker inspect api`	Config as JSON
`docker stats`	Live CPU, memory
`docker cp api:/app/main.py .`	Copy a file out
`docker rm -f api`	Force-remove

Core concepts

TERM	WHAT IT IS
Image	Read-only template: OS layer + dependencies + your code.
Container	A running instance of an image. Disposable by design.
Registry	Where images live (Docker Hub, GHCR). `pull` / `push`.
Volume	Storage that outlives the container.
Network	Lets containers reach each other by name.
Compose	One YAML file that runs a multi-container app.

docker run flags

FLAG	DOES
`-d`	Run in the background (detached)
`-it`	Interactive terminal
`-p 8000:8000`	Publish **host:container** port
`-v data:/data`	Mount a volume or folder
`-e KEY=value`	Set an environment variable
`--env-file .env`	Load variables from a file
`--name api`	Give the container a name
`--rm`	Delete the container on exit
`--network net`	Attach to a network
`--memory 2g`	Cap memory
`--gpus all`	Expose NVIDIA GPUs
`--restart unless-stopped`	Auto-restart policy

Cleanup

RECLAIM DISK

`docker system df`	Disk usage
`docker container prune`	Remove stopped
`docker image prune`	Remove dangling
`docker image prune -a`	Remove all unused
`docker builder prune`	Clear build cache
`docker volume prune`	Deletes volume data
`docker system prune`	Broad cleanup

Copyright © Codebasics Inc. All rights reserved.

Dockerfile instructions

INSTRUCTION	DOES
FROM	Base image. Use -slim variants.
WORKDIR	Set the working directory (creates it).
COPY	Copy files from build context into the image.
RUN	Run a command at **build** time. Creates a layer.
ENV	Set an environment variable, kept at runtime.
ARG	Build-time variable only.
EXPOSE	Documents the port. Does not publish it.
USER	Run as a non-root user.
CMD	Default command at **run** time. Easy to override.
ENTRYPOINT	Fixed executable. CMD becomes its args.
HEALTHCHECK	Command that tells Docker the app is healthy.

Dockerfile: the fast-rebuild pattern

STEP 2

```
FROM python:3.12-slim
WORKDIR /app

# 1. deps first: this layer is cached
COPY requirements.txt .
RUN pip install -r requirements.txt

# 2. code last: changes often
COPY . .

CMD ["uvicorn", "main:app",
  "--host", "0.0.0.0", "--port", "8000"]
```

Config and secrets

WHERE	USE IT FOR
ENV in Dockerfile	Safe defaults baked into the image
-e KEY=value	One-off override at run time
--env-file .env	Many variables, kept out of the image
environment: in Compose	Wiring between services
env_file: in Compose	Load a .env file (required: false if optional)

> Never COPY a .env file or bake API keys into an image. Anyone with the image can read them. Keep .env in .dockerignore and .gitignore.

Layer caching

STEP 2

- Docker rebuilds from the **first changed layer** down. Put what changes least at the top.
- Copy the dependency file, install, **then** copy the code.
- Use slim base images and --no-cache-dir for pip.
- Add a .dockerignore so the build context stays small.

```
# .dockerignore
.venv
__pycache__
.env
.git
```

```
# BuildKit cache mount (uv)
RUN --mount=type=cache,target=/root/.cache/uv \
  uv sync --frozen --no-dev
```

Multi-stage builds

STEP 4

```
# Stage 1: build and test
FROM python:3.12-slim AS builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --prefix=/install \
  --no-cache-dir -r requirements.txt
COPY . .
RUN pytest test_main.py -v

# Stage 2: lean runtime image
FROM python:3.12-slim AS runtime
WORKDIR /app
COPY --from=builder /install /usr/local
COPY main.py .
CMD ["uvicorn", "main:app",
  "--host", "0.0.0.0", "--port", "8000"]
```

> A failing test fails the build, so a broken image never ships. Dev tools and test files stay out of the final image.

Tags and registries

- Name format: registry/user/name:tag. No tag means latest.
- latest is just a default name, not the newest build. Pin versions like api:v1.
- Pin base images too: python:3.12-slim, not python.
- Log in once with docker login, then tag and push.

Copyright © Codebasics Inc. All rights reserved.

Volumes

STEP 3

TYPE	USE IT FOR
Named volume `-v data:/data`	Databases and app state. Docker manages it.
Bind mount `-v ./src:/app`	Live code editing from a host folder.
tmpfs `--tmpfs /tmp`	Scratch data kept in memory only.

`docker volume ls`	List volumes
`docker volume inspect data`	Details and users
`docker volume rm data`	Delete a volume

> A container's own filesystem is deleted with the container. Anything you need to keep goes in a volume.

Networking

STEP 5

- On a Compose network, reach other services by **service name**: `redis:6379`.
- `localhost` inside a container means **that container**, not your machine.
- Publish ports (`-p`) only for what the browser needs. Databases stay internal.
- Reach the host from a container: `host.docker.internal`.
- Bind apps to `0.0.0.0`, not `127.0.0.1`, or the port mapping won't work.

`docker network ls`	List networks
`docker network create appnet`	Create one

When it breaks

SYMPTOM	CHECK
Port already allocated	Change the host side: `-p 8001:8000`
Container exits at once	`docker logs`. Is the `CMD` correct?
Browser can't reach the app	App must listen on `0.0.0.0`
Can't connect to Redis or DB	Use the service name, not `localhost`
Data gone after restart	Mount a named volume
Exit code 137	Out of memory. Raise `--memory`.
Rebuilds take forever	Reorder layers. Add `.dockerignore`.

Docker Compose

STEP 5

```yaml
name: hello-api
services:
 redis:
  image: redis:7-alpine
  volumes:
   - redis-data:/data
 api:
  build: .
  ports:
   - "${API_PORT:-8000}:8000"
  environment:
   - REDIS_HOST=redis
  depends_on:
   - redis
volumes:
 redis-data:
```

Compose commands

`docker compose up --build`	Build and start
`docker compose down`	Stop, keep volumes
`docker compose down -v`	Also delete volumes
`docker compose ps`	Service status
`docker compose logs -f api`	Follow one service
`docker compose exec api sh`	Shell in a service
`docker compose run --rm api <cmd>`	One-off command

> **depends_on** controls start **order**, not readiness. Add a **healthcheck** with **condition: service_healthy** when the app needs the database ready.

GPU override file

STEP 6

```yaml
services:
 embeddings:
  deploy:
   resources:
    reservations:
     devices:
      - driver: nvidia
        count: all
        capabilities: [gpu]
```

> Save as **compose.gpu.yaml** and merge it: **docker compose -f compose.yaml -f compose.gpu.yaml up --build**. Needs an NVIDIA GPU and the NVIDIA Container Toolkit.

Copyright © Codebasics Inc. All rights reserved.